CS459/698 Privacy, Cryptography, Network and Data Security
Assignment Three

Please use Piazza for questions and clarifications.

Assignment due date: Jul 29, 2026 at 3:00 pm

Total Marks: 70 (38 written + 32 programming)

TA: Alim Dhanani

TA Office hours: Mondays 1:30 – 2:30 pm, DC 3333A (CrySP Lab office hours room)

- Submissions are to be uploaded to the appropriate LEARN dropbox as a zipped folder containing both the written and programming components.
  - Please see the Submission Instructions section for more details
- All questions (about the programming and written parts) should be asked privately on Piazza under Assignment 3.
- The TA may decide to make responses to your private questions public so that everyone benefits from knowing the same information. Questions where you need to post partial solutions or questions that describe the locations of vulnerabilities or code to exploit those vulnerabilities will be sanitized before the TA provides a public response.

Written (38 marks)

1. Syntactic Notions of Privacy (9 marks)

Simple redaction

University of Waterloo is considering the release of anonymized grades data for their courses. CS500 is a course with 7 students. Program (grad, undergrad) and sex (M, F) are quasi-identifiers. Financial assistance (OSAP, non-OSAP) and grades are confidential attributes. The data is shown below in Table 1. For statistics with only one individual, UW suppresses (redacts) the information to preserve their privacy.

Statistic	Group	Count	Median	Mean
1A	Total	7	76	76
2A	Female	3	76	78
2B	Male	4	73	74.5
2C	Grad	3	88	82
2D	Undergrad	4	67	71.5
3A	OSAP	3	58	70
3B	Non-OSAP	4	82	80.5
4A	Male Undergrad	3	58	70
4B	Male Grad	**	**	**

Table 1: Anonymized CS500 statistics. ** represents redacted data.

(a) **(2 marks)** Find the grade of the redacted Male Grad student. Show your reasoning.

Higher k-values

In the simple example above, the dataset was k-anonymous with $k = 1$, since there was a group with only one record. We can see that suppressing rows that are k-anonymous with $k = 1$ may not completely preserve the privacy of our data. Let's look at another dataset and implement higher k values.

You have heard about a mysterious data security company called **Hash'em Labs**, which you are eager to join as a **co-op student**. Before facing a series of challenging interviews, you want to ensure that their salary range aligns with your expectations.

Hash'em Labs publicly shares salary data for their currently-employed **co-op students** they have hired from the **University of Waterloo** on their website. However, to maintain privacy, the data is presented in a **pseudo-anonymous** format.

The dataset shared on the company's website is shown in Table 2, which includes[1]:

- **ID**: A pseudo-random identifier for each student.
- **Age**: The student's age.
- **Major**: The student's field of study (**CS** - Computer Science, **CE** - Computer Engineering, **EE** - Electrical Engineering, **Finance** - Finance).
- **Degree**: The student's degree type (**Bachelor, Master, PhD**).
- **Salary***: The monthly co-op salary in Canadian dollars.

[1]The sensitive columns are shown by *.

ID	Age	Major	Degree	Salary*
101	18	CS	Bachelor	5800
102	18	CS	Bachelor	6000
103	20	CS	Bachelor	6200
104	22	CS	Master	5700
105	22	CS	Master	5900
106	24	CS	Master	6100
107	25	CS	PhD	7000
108	25	CS	PhD	7200
109	27	CS	PhD	7400
110	18	CE	Bachelor	6000
111	18	CE	Bachelor	6300
112	20	CE	Bachelor	6500
113	22	CE	Master	7200
114	22	CE	Master	7500
115	24	CE	Master	7700
116	25	EE	PhD	8200
117	25	EE	PhD	8500
118	27	EE	PhD	8700
119	23	EE	Master	7800
120	23	EE	Master	8000
121	24	EE	Master	8300
122	19	Finance	Bachelor	6400
123	19	Finance	Bachelor	6700
124	20	Finance	Bachelor	6900

Table 2: Anonymized salary dataset from Hash'em Labs regarding UWaterloo students

Answer the following questions:

(b) **(2 marks)** Identify the attributes forming a *quasi-identifier* in this table. Should `ID` be considered a quasi-identifier? Justify your answer.

(c) **(1 marks)** Based on your selected quasi-identifier, determine whether Table 2 satisfies k-anonymity and find the value of k.

(d) **(2 marks)** Based on the methods discussed in the lectures, which approach would be most effective for modifying the `Age` column to achieve $k = 3$ while maximizing information retention? Briefly state a specific example of the modifications to the Age column that satisfies $k = 3$.

(e) **(2 marks)** After increasing k to 3 in Table 2, assess whether the table satisfies l-diversity. Begin by defining l-diversity, determine the corresponding value of l, and justify why this value is accurate based on the data.

2. Differential Privacy (11 marks)

Beverly Hills Feline Hospital maintains a relational dataset D shown in Table 3, which contains information about the hospital's feline customers.

Name	Gender	Colour	Age	Weight (kg)	Spayed
Lily	Female	Orange	2	3.5	Yes
Gracie	Female	Blue	5	4.0	Yes
Luna	Female	Tuxedo	11	2.8	Yes
Finlay	Male	Tuxedo	7	5.2	Yes
Max	Male	Black	3	3.9	No
Lola	Female	Grey	4	4.4	No
Angel	Female	Black	6	4.8	Yes
Yoshi	Male	Tuxedo	2	5.2	No

Table 3: Dataset of feline patients

To promote cats' privacy, the hospital would like to publish a data sample with **differential privacy**. The hospital guarantees every cat's weight lies in $[0, 10]$ kg.

Recall the Laplace mechanism from lecture 18:

- $Y \sim Lap(\mu, b)$, $\mu = 0$
- PDF: $p(y) = \frac{1}{2b} e^{-|y-\mu|/b}$
- Tail probability: $Pr[|Y| > t] = e^{-t/b}$

(a) **(2 marks)** Let $f_1(D) =$ num. spayed cats. What is the sensitivity of this query under both **unbounded** definitions (adding or removing one row) and **bounded** definitions (changing one row)?

(b) **(2 marks)** Let $f_2(D) =$ mean weight of all cats. What is the bounded sensitivity of this query?

(c) **(1 mark)** The clinic wants to publish $f_1(D)$, the number of spayed cats. If they set the privacy parameter $\epsilon = 0.25$, what is the value of the Laplace scale parameter b using unbounded DP?

(d) **(1 mark)** Suppose the noise actually drawn is $y = 3.2$. What value is released? (Round to 1 decimal place.)

(e) **(2 marks)** Calculate $Pr[|Y| > 2]$ for this mechanism, i.e. the probability the noise has magnitude greater than 2.

(f) **(3 marks)** A client is concerned the privacy parameter ϵ is not private enough, they suggest $\epsilon = 0.05$. Will the released data with this parameter be useful? Justify. Include a brief example.

3. Private Information Retrieval (11 marks)

Alice is a user tired of seeing targeted advertisements in every service she uses. There's a new online shopping company called BuyNoSpy that wants to offer privacy-friendly shopping.

BuyNoSpy's inventory of items for sale is presented in the form of an $n \times m$ matrix A. Each element of A is a single bit. Row i corresponds to the product with ID i. Each row of the matrix contains m bits that encode properties of the product.

Their competitors use a simple non-private scheme for a user to retrieve information about a product. The scheme for a product indicated by row c of the matrix A is as follows: the user sends the row number c to the server. The server returns row c of the matrix A to the user. Maliciously, the server would log the c that each user sent and build a detailed profile of their search history.

Note that sending a row number c to the server is mathematically equivalent to left-multiplying matrix A by a unit row vector of length n with a 1 in index c (that is, a vector with 0s as its elements in all indices other than index c). This vector is known as the query vector. So, from here on, we describe the original scheme as follows: the user sends a query vector $\vec{q}$ and the server computes $\vec{q} \cdot A$. That is, the server multiplies the query vector with the inventory matrix. Finally, the server sends the resulting row to the user.

So, for $c = 2$, and a 3×2 matrix A, we get:

$$\vec{q} \cdot A = [0 \;\; 1 \;\; 0] \cdot \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \\ A_{31} & A_{32} \end{bmatrix} = [A_{21} \;\; A_{22}]$$

Note: Addition operations as part of matrix multiplication are performed bitwise modulo 2; i.e., the additions are effectively XOR operations.

BuyNoSpy aims to **improve** upon this original and non-private scheme and is testing out **new schemes**.

We define:

- **Upload complexity**: The total number of bits the user transmits to BuyNoSpy while making a query. In the original scheme, this value is n.
- **Download complexity**: The total number of bits BuyNoSpy sends back to the user in response to a query. In the original scheme, this value is m.
- **Total complexity**: The overall communication cost, defined as the sum of upload and download complexities.

(a) **(1 mark)** Find the most **trivial** solution that ensures Alice retrieves $\vec{q} \cdot A$ without BuyNoSpy learning $\vec{q}$.

Hint: This solution should not involve cryptography and may have the highest amount of communication complexity.

(b) **(2 marks)** Calculate the total communication complexity of your solution from part (a) and provide a justification for your result. Additionally, determine the probability that the server correctly guesses the intended row, c, and explain your reasoning.

(c) **(2 marks)** BuyNoSpy is concerned about its bandwidth usage, so it wants to count the total number of matrix elements that its customers download. Given the two following settings:

(i) Alice wants to obtain the details of $\frac{n}{2}$ products.

(ii) $\frac{n}{2}$ customers at different locations want to obtain the details of one product each.

Calculate the total number of matrix elements[2] downloaded using your trivial scheme in each setting. In which setting does the trivial scheme involve downloading a smaller number of matrix elements? Explain your reasons why.

BuyNoSpy's accountant is pulling out their hair trying to fund the internet bills sustaining the trivial scheme introduced in the previous question. To mitigate this, BuyNoSpy adopts a secure MPC[3] approach called 3-server information-theoretic PIR (IT-PIR).

BuyNoSpy has negotiated with two other companies; each company will host a server that contains an identical copy of A (now we have a total of three servers). IT-PIR requires assuming that each of these servers does not share the requests that they received, or the responses that they will send back, and so you may make this non-collusion assumption. Using these 3 servers, BuyNoSpy proposes the following solution for Alice to retrieve row c:

- As usual, the user generates a query vector $\vec{q}$ that is 1 at element c and 0 otherwise, but the user doesn't directly send $\vec{q}$ to any of the 3 servers.
- The user generates two uniformly random bit vectors $\vec{q_1}$ and $\vec{q_2}$, of size n (Each bit is flipped at random for each element of each vector).
- The user generates a vector $\vec{q_3}$ by XOR'ing each element from $\vec{q_1}$ and $\vec{q_2}$ and $\vec{q}$. That is, $\vec{q_3}$ is obtained using the following formula in which $\oplus$ represents the XOR operator.

$$\vec{q_3} = \vec{q_1} \oplus \vec{q_2} \oplus \vec{q}$$

- The user uploads $\vec{q_1}$ to server 1, $\vec{q_2}$ to server 2 and $\vec{q_3}$ to BuyNoSpy's original server.
- Each server computes the following for the received query: $\vec{s_i} = \vec{q_i} \cdot A$ and returns it to the user, following the usual matrix multiplication operation with modulo 2 additions as discussed earlier.
- The user computes $\vec{s} = \vec{s_1} \oplus \vec{s_2} \oplus \vec{s_3}$.

(d) **(3 marks)** Show that at the end of the protocol, the user obtains row c of matrix A. That is, $\vec{s} = \vec{q} \cdot A = A_c$. You may express your proof using words or mathematical expressions, but be sure to give a detailed explanation.

(e) **(2 marks)** In the above solution provided by BuyNoSpy, do the servers learn any information about the desired row? If yes, explain what they learn, or explain why not.

(f) **(1 mark)** What is the total number of vector elements that are uploaded and downloaded in BuyNoSpy's IT-PIR solution?

[2] a matrix element is an individual entry of the matrix A

[3] Multi-party computation – Wikipedia

4. Blockchain Theory (7 marks)

A **Merkle tree** is a binary tree of hashes where each leaf is the hash of a transaction and each internal node is the hash of its two children. The root hash (Merkle root) is stored in the block header. An example of a Merkle tree is shown in Figure 1.

(a) **(1 mark)** Why use a Merkle tree rather than hashing all transactions as a single concatenated string?

(b) **(2 marks)** Suppose we want to create a Merkle tree using the 8 ordered data blocks shown in the list below. What would be the root hash of this Merkle tree?

- 63
- 79
- 47
- 74
- 70
- 51
- 64
- 91

For simplicity, let's define the hash function $H(x) =$ the sum of the digits in x . For example, $H(12) = 3$ and $H(99) = 18$. Let us define $x + y =$ concat$(x, y) = x||y$, for example $12 + 34 = 1234$. This means that $H(x, y) = H(x + y)$, as in $H(12, 34) = H(12 + 34) = H(1234) = 10$.

(c) **(2 marks)** In *selfish mining*, a miner who discovers a new block withholds it and continues mining on top of it privately. Explain the advantage this gives the selfish miner over honest miners.

(d) **(2 marks)** In proof-of-stake (PoS), a selected validator can propose multiple conflicting blocks at negligible cost (the *nothing-at-stake* problem). Explain why this does not arise in proof-of-work, and describe one mechanism PoS protocols use to mitigate it.

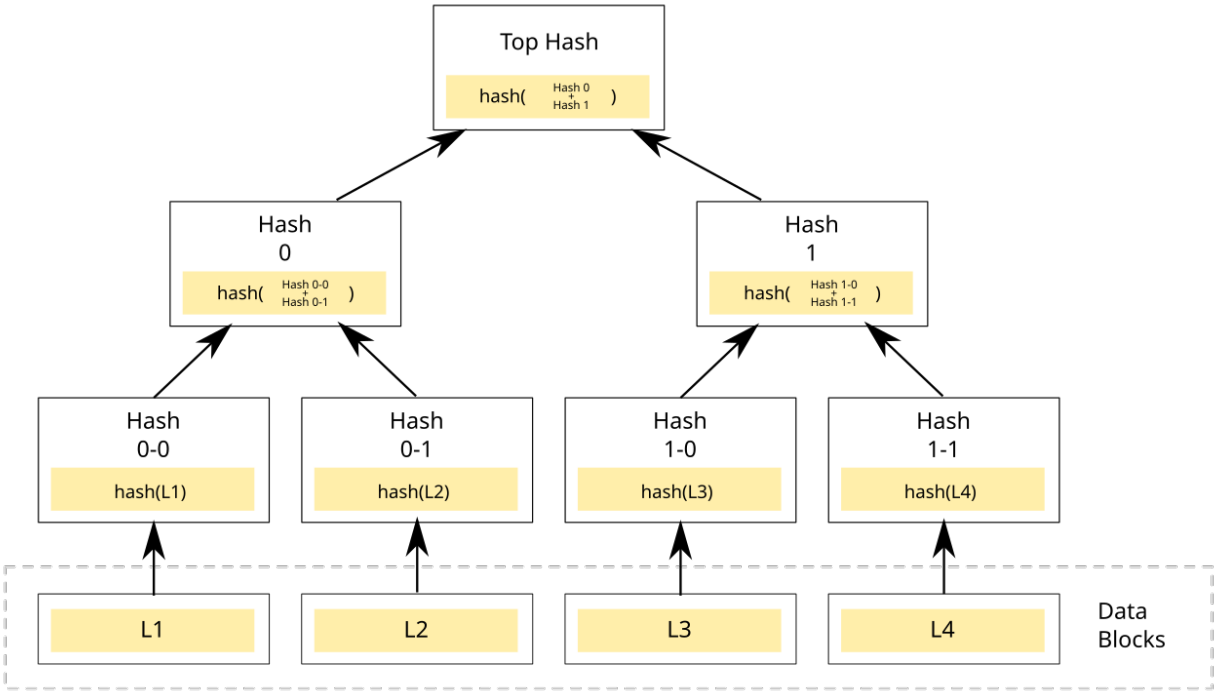


Figure 1: Merkle tree mechanism. (Source: Wikipedia)

Programming (32 marks)

5. Blockchain (17 marks)

You've just joined the cutting-edge blockchain company *Hash'em Labs* on a co-op program. The company is testing out a new cryptocurrency *GooseCoin* made only for uWaterloo students, and needs your help in developing it.

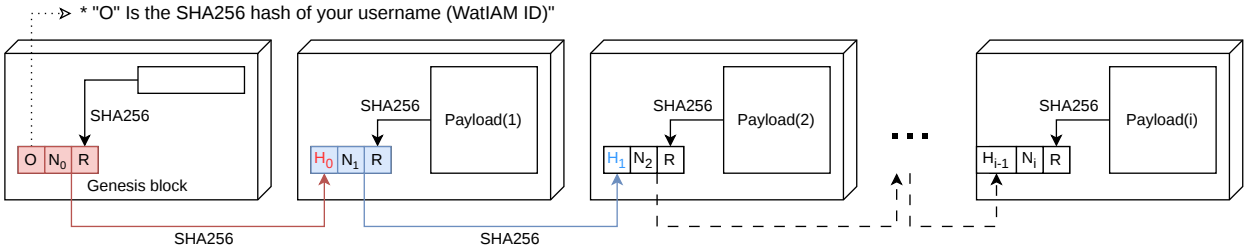


Figure 2: *GooseCoin*'s blockchain structure.

(a) (12 marks) Finish the source code and mine a chain.

The previous co-op student only developed a half-finished Python file (`A3Q5.py`), alongside a dataset (`transactions.csv`).

Time to start mining! Using your (lowercase) WatIAM as a seed in the genesis block, mine all the blocks with transactions from the dataset, and provide your dump file: run `A3Q5.py` with your WatIAM, hash length of 7, and a block size limit of 25,000.

(b) (5 marks) Find a collision for the last block you mined.

Your boss wants to test the "robustness" of *GooseCoin*. She wants you to find a collision for the last block you mined! The previous co-op set up the blockchain using truncated hash values for "storage efficiency", which *probably* (most definitely) creates a vulnerability.

Your boss gives you a sample of **fraudulent transactions** in the file `fraudulent.csv`. She asks you to construct a **collision block** that satisfies the following:

- Uses the same `previous_hash` as your last mined block
- Hashes to the same value as your last mined block
- Has a different `nonce` than your last mined block
- Contains the transactions from `fraudulent.csv` as its payload, instead of the original transactions
- Using the same block size limit as in part (a)

Find this collision block and send its dump file to your boss. Figure 3 shows this collision.

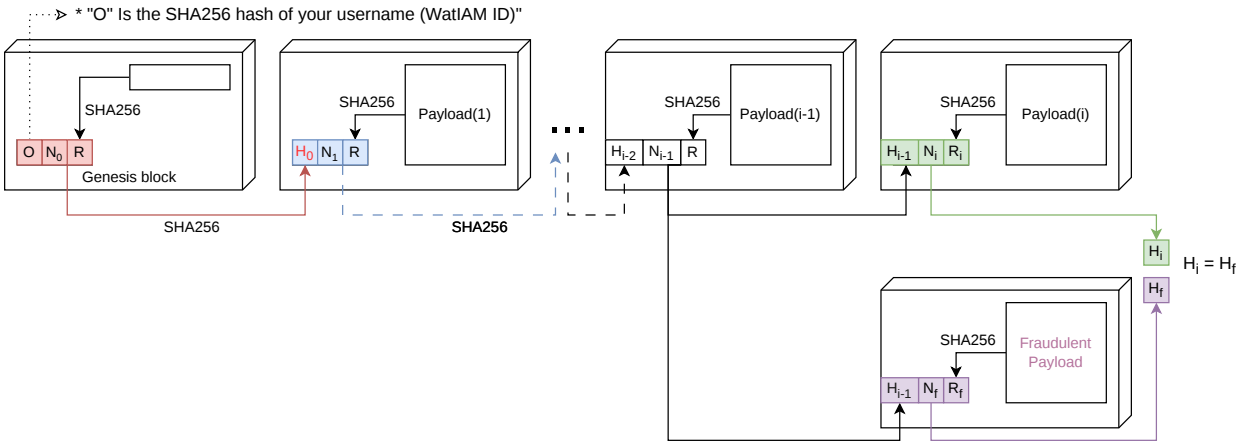


Figure 3: Your boss wants you to find a collision for block i, your last mined block, using fraudulent transactions as the payload.

Instructions.

You are given **Figures** 2 and 3 along with a detailed set of instructions to guide you through the process:

- **Hashing Algorithm:** Use the `SHA256` hashing algorithm for all hashing operations. It will be truncated to the first `hash_length` characters.

- **Dataset:** The file `transactions.csv` contains the following columns:
  - `Sender`: The sender node's address.[4]
  - `Receiver`: The receiver node's address.
  - `Transaction Amount`: The amount of cryptocurrency sent.
  - `Transaction Fee`: The fee received by the miner for processing the transaction.
  - `Transaction Size`: The size of the transaction in `bytes`.

- **Adjustable Parameters:** Your program must dynamically allow adjustments for the following parameter:
  - `block_size_limit`: The maximum allowable block size in bytes.[5]

- **Global variables:**
  - `hash_length`: All block hashes are truncated to the first `hash_length` characters before use. This value is set by command line parameter and is fully handled by the starter code.
  - `leading_zeroes = (hash_length - 2)`: The **hardness** to mine a block: the required number of leading `0`s in the block hash[6]

[4]Most cryptocurrencies use wallet addresses, *GooseCoin* uses WatIAM usernames.
[5]For simplicity, we ignore the size of other parameters in the block header and only consider the transaction sizes. Ensure that you fully utilize the block space.
[6]Larger values significantly increase computational time.

- **Blockchain Structure:** The blockchain consists of a particular number of blocks. Each block contains:
  - **Payload:** A set of selected transactions.
  - **Header:** Includes the following fields:
    * **H:** `previous_hash` – A truncated SHA256 hash of the previous block's header. The genesis block uses the truncated hash of your username (WatIAM ID).
    * **N:** `nonce` – A natural number used for mining.
      Note: To maintain consistency, start from 1 and increment sequentially to find the **smallest valid nonce** that meets the `hardness` requirement.
    * **R:** The truncated SHA256 hash of the payload.

- **Transaction Selection Criteria:** Transactions must be selected and added to blocks based on greedy selection: sequentially go through the transactions in the dataset and add them until no more transactions can be added to that block — i.e. the next transaction will make the sum of the block's `Transaction Size` to exceed `block_size_limit`. The transactions in the dataset are already pre-sorted.
  - Ensure that the total transaction size in a block does not exceed `block_size_limit`.

- **Building the Payload:** To format the list of transactions into a standard string payload:
  - Use the `Transaction` class to create transaction objects (the string representation is already defined for you in the template.).
  - Select transactions according to the criteria above.
  - Use the function `stringify_payload` to construct the payload.

- **Dump files:** When your chain is fully mined, your program will produce `blockchain.txt` via `dump_nonces()`. This file has the following format:
  - Line 1: Your WatIAM ID
  - Lines 2+: The nonce of each mined block, one per line, in order (genesis block excluded)

  For Part (b), your program will also produce `collision.txt` via `dump_collision()`. This file has the following format:
  - Line 1: Your WatIAM ID
  - Line 2: The nonce of the collision block

  The grader will use these files to reconstruct and verify your blockchain.

- **Command Line Interface (CLI):** Your program should accept the following CLI command:

```
 A3Q5.py <WatIAM> <path_to_transaction.csv> <hash_length> <block_size_limit>
 # Generates blockchain.txt containing the blockchain dump.
 # Generates collision.txt, containing the dump of the fraudulent block

 # Example usage:
 A3Q5.py hasher666 './datasets/transaction.csv' 7 25000

```

To submit:

- Your completed `A3Q5.py`
- `blockchain.txt` and `collision.txt` using your WatIAM, hash length of 7 and a block size limit of 25,000

Important Notes:

- The provided `A3Q5.py` file contains a **skeleton code** that serves as a foundation for your implementation.
- Complete the missing parts (marked as TODO)
- Ensure your WatIAM is passed into the program **lowercase**
- Don't forget to use the smallest possible nonce for each block
- Do **not** use any external libraries beyond those already imported in the script
- Before submission, refer to the `README.md` file in the root directory for detailed instructions on testing your code locally. This step ensures your implementation functions correctly and adheres to the required standards.

6. Membership Inference Attacks (15 marks)

You are tasked with launching a Membership Inference Attack (MIA) on a highly confidential model. This is a powerful attack that attempts to infer whether a particular data point was part of the model's training set.

Your objective is to analyze the model's behaviour and identify which inputs were likely part of its training set (i.e., members).

You are provided with the following files:

- Starter code `A3Q6.py`
- A trained model file: `trained_model.pth`
- A dataset: `final_dataset_membership.csv`, containing:
  - `index`: unique identifier (do not change this)
  - `pixels`: a 28×28 grayscale image flattened into space-separated integers
  - `label`: digit class (0–9)

Instructions:

(a) **Update your WatIAM ID:** In the starter code, set the variable `watiam_id = "<your-watiam-id>"` (all lowercase). This will be used to generate a personalized sample of data for evaluation.

(b) **Complete the function** `load_full_dataset()`**:** This function must read the provided CSV file into a pandas DataFrame.

(c) **Implement the function** `predict_membership()`**:** Your goal here is to analyze the model's output confidence and decide whether a sample is a member (1) or non-member (0). Use a threshold or scoring rule and explain your rationale in the report.

(d) **Fill in the report** inside `save_outputs()`**:** The script will generate a Markdown/text file to describe your membership inference strategy. This report must briefly explain how you interpreted the model's behaviour and made membership decisions.

Hints:

- The model performs with very high accuracy on its training data. That often results in much higher confidence on member samples than non-member samples.
- `parse_images()` and `sample_dataset()` are implemented for you. Do **not** modify these functions.

Expected Output: After executing your completed script, it should generate:

- A CSV file named `<watiam-id>_predictions.csv` with two columns: `index` and `is_member`
- `<watiam-id>_approach.md` describing your approach

Tips for Execution:
If setting up locally proves difficult, run the script on Google Colab

- Upload all required files: the Python script, model, and dataset.
- Use `/content/filename` style paths in Colab to access the uploaded files.
- Select GPU under `Runtime > Change` runtime type if needed — the code works on both CPU and GPU.

Restrictions:

- Do not use external libraries other than the ones already imported in the starter script.
- Do not modify any of the helper function signatures or sampling code.

Submission Instructions

Your submission must strictly follow the instructions below:

- Upload your work to the appropriate LEARN dropbox as a single zipped folder, named exactly: `<your-watiam-id>_A3.zip`. This archive must contain both the written and programming components.
- **Do not include an extra parent folder inside the zip archive**. The root of your zip file should directly contain the following:
  - `A3_written.pdf`: your answers to the written questions.
  - A folder named `Q5`, containing:
    * `A3Q5.py`
    * `blockchain.txt`
    * `collision.txt`
  - A folder named `Q6`, containing:
    * `A3Q6.py`
    * `<watiam-id>_predictions.csv`
    * `<watiam-id>_approach.md`